

# General Sam

## 广义后缀自动机

### 前置知识

广义后缀自动机基于下面的知识点

- [字典树 \(Trie 树\)](#)
- [后缀自动机](#)

请务必对上述两个知识点非常熟悉之后，再来阅读本文，特别是对于 **后缀自动机** 中的 **后缀链接** 能够有一定的理解

### 引入

#### 起源

广义后缀自动机是由刘研绎在其 2015 国家队论文《后缀自动机在字典树上的拓展》上提出的一种结构，即将后缀自动机直接建立在字典树上。

大部分可以用后缀自动机处理的字符串的问题均可扩展到 Trie 树上。——刘研绎

#### 约定

参考 [字符串约定](#)

字符串个数为  $n$  个，即

约定字典树和广义后缀自动机的根节点为  $0$  号节点

#### 概述

后缀自动机 (suffix automaton, SAM) 是用于处理单个字符串的子串问题的强力工具。

而广义后缀自动机 (General Suffix Automaton) 则是将后缀自动机整合到字典树中来解决对于多个字符串的子串问题

## 常见的伪广义后缀自动机

1. 通过用特殊符号将多个串直接连接后，再建立 SAM
2. 对每个串，重复在同一个 SAM 上进行建立，每次建立前，将 `last` 指针置零

方法 1 和方法 2 的实现方式简单，而且在面对题目时通常可以达到和广义后缀自动机一样的正确性。所以在网络上很多人会选择此类写法，例如在后缀自动机一文中最后一个应用，便使用了方法 1 [\(原文链接\)](#)

但是无论方法 1 还是方法 2，其时间复杂度较为危险

## 构造广义后缀自动机

根据原论文的描述，应当在多个字符串上先建立字典树，然后在字典树的基础上建立广义后缀自动机。

### 字典树的使用

首先应对多个串创建一棵字典树，这不是什么难事，如果你已经掌握了前置知识的前提下，可以很快的建立完毕。这里

为了统一上下文的代码，给出一个可能的字典树代码。

## ► 实现

这里我们得到了一棵依赖于 `next` 数组建立的一棵字典树。

## 后缀自动机的建立

如果我们把这样一棵树直接认为是一个后缀自动机，则我们可以得到如下结论

- 对于节点  $i$ ，其  $len[i]$  和它在字典树中的深度相同
- 如果我们对字典树进行拓扑排序，我们可以得到一串根据  $len$  不递减的序列。BFS 的结果相同

而后缀自动机在建立的过程中，可以视为不断的插入  $len$  严格递增的值，且差值为 1。所以我们可以将对字典树进行拓扑排序后的结果做为一个队列，然后按照这个队列的顺序不断地插入到后缀自动机中。

由于在普通后缀自动机上，其前一个节点的  $len$  值为固定值，即为  $last$  节点的  $len$ 。但是在广义后缀自动机中，插入的队列是一个不严格递增的数列。所以对于每一个值，对于它的  $last$  应该是已知而且固定的，在字典树上，即为其父亲节点。

由于在字典树中，已经建立了一个近似的后缀自动机，所以只需要对整个字典树的结构进行一定的处理即可转化为广义后缀自动机。我们可以按照前面提出的队列顺序来对整个字典树上的每一个节点进行更新操作。最终我们可以得到广义后缀自动机。

对于每个点的更新操作，我们可以稍微修改一下 SAM 中的插入操作来得到。

对于整个插入的过程，需要注意的是，由于插入是按照  $len$  不递减的顺序插入，在进行 `clone` 后的数据复制过程中，不可以复制其  $len$  小于当前  $len$  的数据。

## 过程

根据上述的逻辑，可以将整个构建过程描述为如下操作

1. 将所有字符串插入到字典树中
2. 从字典树的根节点开始进行 BFS，记录下顺序以及每个节点的父亲节点
3. 将得到的 BFS 序列按照顺序，对每个节点在原字典树上进行构建，注意不能将  $len$  小于当前  $len$  的数据进行操作

## 对操作次数为线性的证明

由于仅处理 BFS 得到的序列，可以保证字典树上所有节点仅经过一次。

对于最坏情况，考虑字典树本身节点个数最多的情况，即任意两个字符串没有相同的前缀，则节点个数为  $\sum_{i=1}^n |s_i|$ ，即所有的字符串长度之和。

而在后缀自动机的更新操作的复杂度已经在 [后缀自动机](#) 中证明

所以可以证明其最坏复杂度为线性

而通常伪广义后缀自动机的平均复杂度等同于广义后缀自动机的最差复杂度，面对对于大量的字符串时，伪广义后缀自动机的效率远不如标准的广义后缀自动机

## 实现

对插入函数进行少量必要的修改即可得到所需要的函数

## ► 参考代码

- 由于整个 BFS 的过程得到的顺序，其父节点始终在变化，所以并不需要保存  $last$  指针。
- 插入操作中，`int cur = next[last][c];` 与正常后缀自动机的 `int cur = tot++;` 有差异，因为我们插入的节点已

经在树型结构中完成了，所以只需要直接获取即可

- 在 `clone` 后的数据拷贝中，有这样的判断 `next[clone][i] = len[next[q][i]] != 0 ? next[q][i] : 0`；这与正常的后缀自动机的直接赋值 `next[clone][i] = next[q][i]`；有一定差异，此次是为了避免更新了 `len` 大于当前节点的值。由于数组中 `len` 当且仅当这个值被 BFS 遍历并插入到后缀自动机后才会被赋值

## 性质

- 广义后缀自动机与后缀自动机的结构一致，在后缀自动机上的性质绝大部分均可在广义后缀自动机上生效（[后缀自动机的性质](#)）
- 当广义后缀自动机建立后，通常字典树结构将会被破坏，即通常不可以用广义后缀自动机来解决字典树问题。当然也可以选择准备双倍的空间，将后缀自动机建立在另外一个空间上。

## 应用

### 所有字符中不同子串个数

可以根据后缀自动机的性质得到，以点 为结束节点的子串个数等于

所以可以遍历所有的节点求和得到

例题：[【模板】广义后缀自动机 \(广义 SAM\)](#)

► 参考代码

### 多个字符串间的最长公共子串

我们需要对每个节点建立一个长度为 的数组 `flag`（对于本题而言，可以仅为标记数组，若要求出此子串的个数，则需要改成计数数组）

在字典树插入字符串时，对所有节点进行计数，保存在当前字符串所在的数组

然后按照 `len` 递减的顺序遍历，通过后缀链接将当前节点的 `flag` 与其他节点的合并

遍历所有的节点，找到一个 `len` 最大且满足对于所有的 `k`，其 `flag` 的值均为非 的节点，此节点的 即为解

例题：[SPOJ Longest Common Substring II](#)

► 参考代码

---

本页面最近更新：2024/10/9 22:38:42，[更新历史](#)

发现错误？想一起完善？[在 GitHub 上编辑此页！](#)

本页面贡献者：[do-while-true](#), [Enter-tainer](#), [Hukeying](#), [iamtwz](#), [ImpleLee](#), [lr1d](#), [kenlig](#), [ksyx](#), [ouuan](#), [shuzhouliu](#), [Tiphereth-A](#)

本页面的全部内容在 [CC BY-SA 4.0](#) 和 [SATA](#) 协议之条款下提供，附加条款亦可能应用